# Laboratory Research Problem 04 (LRP04)

CMSC 180: Introduction to Parallel Computing

Charles Andrei P. De los Reyes
Student Number: 2023-15797
Section: B-3L

April 23, 2026

## Table 1. Time readings after running all instances within one PC on different terminals.

All slaves and the master ran as separate processes on a single PC, communicating through distinct ports on the loopback interface. No core affinity was enforced; the Linux scheduler placed threads freely.

| $n$ | $t$ | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|---|
| 4,000 | 2 | 0.014540 | 0.018172 | 0.018506 | 0.017073 |
| 4,000 | 4 | 0.022927 | 0.022501 | 0.023492 | 0.022973 |
| 4,000 | 8 | 0.027694 | 0.027366 | 0.033009 | 0.029356 |
| 4,000 | 16 | 0.038022 | 0.033686 | 0.033495 | 0.035068 |
| 8,000 | 2 | 0.056518 | 0.060112 | 0.055731 | 0.057454 |
| 8,000 | 4 | 0.085382 | 0.087451 | 0.086881 | 0.086571 |
| 8,000 | 8 | 0.102653 | 0.099831 | 0.111726 | 0.104737 |
| 8,000 | 16 | 0.124436 | 0.122716 | 0.126551 | 0.124568 |
| 16,000 | 2 | 0.204243 | 0.199779 | 0.208155 | 0.204059 |
| 16,000 | 4 | 0.307341 | 0.296739 | 0.303263 | 0.302448 |
| 16,000 | 8 | 0.414202 | 0.394432 | 0.398659 | 0.402431 |
| 16,000 | 16 | 0.433494 | 0.459253 | 0.458044 | 0.450264 |

All times are in seconds.

## Table 2. Time readings after repeating the process, but running all slave instances in a core-affine way.

Each slave process was pinned to a distinct CPU core (cores 1–15) via `sched_setaffinity`; the master remained on core 0. All other conditions matched Table 1.

| $n$ | $t$ | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|---|
| 4,000 | 2 | 0.016576 | 0.015553 | 0.015187 | 0.015772 |
| 4,000 | 4 | 0.023021 | 0.024071 | 0.024345 | 0.023812 |
| 4,000 | 8 | 0.029421 | 0.030629 | 0.029055 | 0.029702 |
| 4,000 | 16 | 0.035423 | 0.034414 | 0.035923 | 0.035253 |
| 8,000 | 2 | 0.052038 | 0.056003 | 0.056865 | 0.054969 |
| 8,000 | 4 | 0.088928 | 0.086846 | 0.085981 | 0.087252 |
| 8,000 | 8 | 0.103423 | 0.101003 | 0.109725 | 0.104717 |
| 8,000 | 16 | 0.130102 | 0.122868 | 0.128082 | 0.127017 |
| 16,000 | 2 | 0.219621 | 0.215163 | 0.225091 | 0.219958 |
| 16,000 | 4 | 0.343269 | 0.333158 | 0.332469 | 0.336299 |
| 16,000 | 8 | 0.414651 | 0.385119 | 0.414849 | 0.404873 |
| 16,000 | 16 | 0.461068 | 0.494783 | 0.489821 | 0.481891 |

# Table 3. Time readings after repeating the process, but running all slave instances on different PCs.

Executed on the ICS Swarm cluster with **hayabusa (10.0.9.134)** as the master and **16 slave drones** — grock, fredrinn, belerick, leomord, khaleed, dyrroth, gusion, lunox, zhuxin, mathilda, natan, balmond, beatrix, alucard, phoveus, freya — each on a separate physical machine over Gigabit Ethernet. Every send and receive in the tree crossed the Ethernet fabric; no loopback shortcut was available, unlike in Tables 1 and 2.

| $n$ | $t$ | Run 1 | Run 2 | Run 3 | Average |
|---|---|---|---|---|---|
| 4,000 | 2 | 7.705485 | 5.440405 | 5.447783 | 6.197891 |
| 4,000 | 4 | 6.820272 | 6.821481 | 6.837200 | 6.826318 |
| 4,000 | 8 | 8.223834 | 8.171413 | 8.177687 | 8.190978 |
| 4,000 | 16 | 9.586386 | 9.621205 | 9.574000 | 9.593864 |
| 8,000 | 2 | 22.989403 | 21.836092 | 21.759297 | 22.194931 |
| 8,000 | 4 | 27.254486 | 27.234979 | 27.228129 | 27.239198 |
| 8,000 | 8 | 32.654327 | 32.708185 | 32.647786 | 32.670099 |
| 8,000 | 16 | 38.553450 | 41.309176 | 41.949142 | 40.603923 |
| 16,000 | 2 | 104.956912 | 106.237788 | 93.714543 | 101.636414 |
| 16,000 | 4 | 111.340261 | 119.479887 | 115.722317 | 115.514155 |
| 16,000 | 8 | 145.615386 | 137.460770 | 141.499060 | 141.525072 |
| 16,000 | 16 | 149.397894 | 148.750442 | 151.504708 | 149.884348 |

# Answers and Observations

## What happened when you ran all slave instances in a core-affine way?

Pinning each slave to its own core (cores 1–15) did not produce a speedup. At the largest problem size, $n = 16,000$, it produced a small slowdown of roughly 7–11% compared

with the non-affine runs in Table 1. The reason is that the workload is I/O-bound, not compute-bound: each slave spends almost all of its time blocked in `recv()` waiting for the kernel, followed by a short `memcpy`. Because there is no real computation between these two steps, letting the Linux scheduler place threads freely is actually more efficient, because it keeps both the kernel's loopback service routines and the user-space receivers on warm caches. Forcing affinity removes that freedom and leaves only core 0 available for the master, its worker threads, and the kernel's loopback service, which becomes a mild bottleneck at large $n$. There is also a small additional overhead from the system call that assigns each slave to its core. Affinity would matter for compute-heavy workloads (where cache locality dominates), but not for pure data distribution.

## What happened when you ran all slave instances on different PCs?

Absolute runtimes grew dramatically, roughly 333× slower at $n = 16,000$, $t = 16$ (from 0.450 s on loopback to 149.9 s on the swarm). This is a bandwidth effect, not a CPU effect: loopback moves several GB/s through RAM, while Gigabit Ethernet is capped near 125 MB/s and is further reduced on the shared swarm fabric. Beyond the raw bandwidth gap, every byte sent across Ethernet also pays the cost of being framed and checksummed in the kernel's TCP/IP stack, copied through socket and NIC buffers, and acknowledged by the receiver, overhead that loopback largely skips. Despite the much larger absolute times, scaling with $t$ stayed logarithmic: going from $t = 2$ to $t = 16$ (an 8× increase in slaves) only increased the time by about 1.47×, which is close to the $\log_2(16)/\log_2(2) = 4\times$ limit imposed by the added tree depth. Run-to-run variance was also higher on the swarm (≈12% spread versus around 4% on a single PC), reflecting competing traffic from other users on the shared network. The key point is that the $\mathcal{O}(\log t)$ advantage of the tree survives on real hardware. It is a property of the algorithm, not an artifact of loopback.

## Is your implementation efficient? Did you use any of the communication techniques discussed in the lecture? If yes, which one?

Yes, the implementation is efficient, and it uses **one-to-many personalized broadcast**, which is one of the four collective techniques covered in the lecture. Each slave receives a *different* submatrix (hence "personalized", not "broadcast"), and the distribution is organized as a recursive split-and-forward (binomial-style) tree rooted at the master.

At each level, the master splits its remaining slaves into two halves, delegates the right half's entire block to the boundary slave (along with instructions to repeat the procedure on the right subtree), and continues recursively on the left half. The recursion ends when each leaf slave receives just its own assigned submatrix. This is what makes the scheme efficient:

- The master initiates only $\lceil \log_2 t \rceil$ parallel sends; for $t = 16$, that is 4 sends instead of 16.
- Every slave that has already received its block becomes a new source, so at depth $d$ the tree has $2^d$ nodes transmitting in parallel.

- Acknowledgments return up the same shape, so the master's ack-wait also finishes in $\mathcal{O}(\log t)$.

The measurements back this up: across all three tables, increasing $t$ by $8\times$ (from 2 to 16) only increases runtime by $1.47\times$–$2.21\times$, far below the $8\times$ a sequential scheme would require. Table 3 shows the cleanest log-depth scaling (only $1.47\times$), because on separate physical machines every depth of the tree uses independent NICs and CPUs.